BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Requirement for an algorithm to work correctly.

Q2. To expose faults on boundaries/unusual inputs.

Q3. A property that stays true after each loop iteration.

Q4. Indexes 0 and 2 sum8; distinct values {4,1,2} count3.

You Do pages · Independent questions

Q1. Start count1 [1]; i1 still1 [1]; i2 still1 [1]; i3→2 [1]; i4→3 [1]; i5 remains3 [1].

Q2. Empty []→0 [2]; equal [5,5]→1 [2]; distinct [1,2,3]→3 [2].

Q3. Algorithm counts transitions and reports3 rather than2 [2]; pre-sort then scan [2]; sorting adds typical O(n log n) time [1], or use set/hash with expected O(n) and space O(n) [1].

Exit Ticket · Model answer

False. Equal values might be separated and overcounted.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

RHS COMPUTING / OCR H446 / mixed LESSON 302 · DRAFT

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

LESSON 302 · Unfamiliar algorithm design and evaluation

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What is a precondition?

Q2. Why write edge-case tests?

Q3. What does a loop invariant express?

Q4. C# array [4,1,4,2]: compute sum at indexes 0 and 2, and number of distinct values.

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

invariant — Condition maintained throughout a loop.

precondition — Assumption required for correctness.

boundary test — Tests around value/size limits.

time complexity — Growth in work as input increases.

New learning

State the precondition — The input must be sorted; equal values then appear consecutively.

Trace the invariant — Start with one distinct value for a nonempty array. Each change from previous value adds one.

Evaluate and test — The algorithm scans once O(n) time, O(1) extra space; test empty, all equal and all distinct inputs.

Worked example

Trace distinct count in [2,2,5,5,7,7,7].

1. Predict first — Duplicates do not increment the counter because each equals the preceding element.

2. Explain the method — Only changes to five and seven increment count; result is three distinct values.

We Do · Practise with guidance

Trace the algorithm on [1,1,1,3,4,4] and state two edge tests.

```
Start1 → at 3 count2 → at 4 count3
Empty []→0; all equal→1
```

Predict first

Three initial ones produce one distinct group; 3 and 4 add two groups.

Explain the method

Test empty input and an all-equal array to confirm base conditions and comparisons.

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Trace DistinctCount on [3,3,3,6,8,8]. Record count after each index. [6]

Question 2

Describe a correct set of test cases for the algorithm and their expected outputs. [6]

Question 3

Explain why the algorithm may fail on unsorted [2,1,2]. Give a repair and complexity consequence. [6]

Exit Ticket · Answer before checking

The distinct-count algorithm is valid for unsorted inputs without modification.